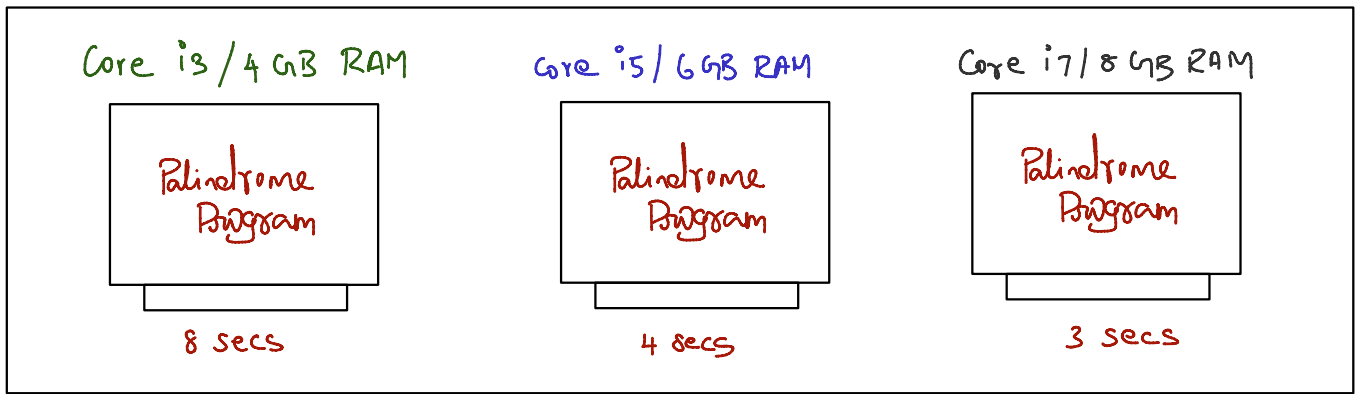


# how to Compare Programs.



- \* Execution Time of the Program depends on Various factors (RAM, ROM, CPU, Surroundings)
- \* We can Compare the Program by using Time and Space Complexity

## Time Complexity:

\* in the worst case, the amount of time taken by an algorithm of Program is called as Time Complexity.

## Space Complexity:

\* in the worst case, the amount of space taken by an algorithm of Program is called as Space complexity.

# Time and Space Complexity

## Introduction:-

Finding no of iterations with

- \* Basic Maths Properties -
- \* Log Basics -
- \* Nested Loops
- \* Arithmetic Progression
- \* Geometric Progression

Big "O" Notations

# Basic Maths Properties :

$[a, b]$  - both  $a$  and  $b$  are included

$(a, b)$  - both  $a$  and  $b$  are excluded

Sum of natural numbers }  $= \frac{n*(n+1)}{2}$

1) how many numbers are there in the range of  $[3, 10]$ ?

3, 4, 5, 6, 7, 8, 9, 10      ans = 8

$$[a, b] = b - a + 1$$

$$10 - 3 + 1 = 8$$

2) how many numbers are there in the range of  $[2, 8]$ ?

2, 3, 4, 5, 6, 7, 8      ans = 7

$$8 - 2 + 1 = 7$$

3) how many numbers are there in the range of  $(3, 10)$ ?

4, 5, 6, 7, 8, 9      ans = 6

$$(a, b) = b - a - 1$$

4) 

```
for(int i=1; i<=n; i++){  
    if(i==n) break;  
}
```

$a=1$     $b=n$   
 $[a, b]$

$[1, n] = n$  iterations

5) 

```
for(int i=1; i<=100; i++){  
    continue;  
}
```

$[1, 100] = 100 - 1 + 1 = 100$  iterations

6) 

```
for(int i=1; i<=n; i++){  
    //statement  
}  
for(int j=1; j<=m; j++){  
    //statements  
}
```

$[1, n] = n$  iterations

+

$[1, m] = m$  iterations

$n+m$  iterations

# Log Basics

$\log_2^{32} = x$  To what value we need to raise  $b$  to get  $a$ .

$$\log_b^a = x \quad b^x = a \quad 2^x = 32 \quad 2^5 = 32$$

$$\log_2^{32} = 5$$

$$\log_2^{128} = 7$$

$$\log_2^{64} = 6$$

$$2^x = 128$$

$$\log_2^{2^5} = 5$$

$$\log_3^{3^7} = 7$$

$$\log_b^{b^x} = x$$

\* how many times we need to divide  $9$  by  $2$  till it reaches  $1$ ?

$$9/2 \xrightarrow{①} 4/2 \xrightarrow{②} 2/2 \xrightarrow{③} 1$$

$$9/8 = 1 \Rightarrow 9/2^3 = 1$$

$$\text{ans} = 3$$

$$\log_2 9 = 3$$

\* how many times we need to divide  $27$  by  $2$  till it reaches  $1$ ?

$$27/2 \longrightarrow 13/2 \longrightarrow 6/2 \longrightarrow 3/2 \longrightarrow 1$$

$$27/16 \Rightarrow 27/2^4 = 1$$

$$\text{ans} = 4$$

$$\log_2^{27} = 4$$

\* how many times we need to divide  $n$  by  $2$  till it reaches  $1$ ?

Proof:

$$n/1 \longrightarrow n/2 \longrightarrow n/4 \longrightarrow n/8 \longrightarrow n/16 \dots \dots n/x = 1$$

$$n/2^0 \longrightarrow n/2^1 \longrightarrow n/2^2 \longrightarrow n/2^3 \dots \dots n/2^x = 1$$

$$n/2^x = 1 \Rightarrow n = 2^x \Rightarrow \text{Take log}$$

$$\Rightarrow \log_2 n = \log_2 2^x$$

$$\Rightarrow x = \log_2 n$$

\* how many times we need to divide 100 by 2 till it reaches 1?

$$\log_2^{100} = 6$$

$$\text{ans} = 6$$

$$\frac{100}{2} \rightarrow \frac{50}{2} \rightarrow \frac{25}{2} \rightarrow \frac{12}{2} \rightarrow \frac{6}{2} \rightarrow \frac{3}{2} \rightarrow 1$$

\* how many times we need to divide 100 by 3 till it reaches 1?

$$\frac{100}{3} \rightarrow \frac{33}{3} \rightarrow \frac{11}{3} \rightarrow \frac{3}{3} \rightarrow 1$$

$$\log_3^{100} = 4 \quad \text{ans} = 4$$

\* 

```
while(n>0){  
    n=n/2;  
}
```

$$n=100 \rightarrow n=50 \rightarrow n=25 \rightarrow n=12 \rightarrow n=6 \rightarrow n=3 \rightarrow n=1$$

$$\text{ans} = \log_2^n \text{ iteration}$$

\* 

```
while(n>0){  
    n=n/4;  
}
```

$$\text{ans} = \log_4 n \text{ iteration}$$

\* 

```
for(int i=1; i<=n; i=i*2){  
    //statements  
}
```

$$n=64$$

|     |     |     |     |      |      |      |
|-----|-----|-----|-----|------|------|------|
| i=1 | i=2 | i=4 | i=8 | i=16 | i=32 | i=64 |
|-----|-----|-----|-----|------|------|------|

$$n \cdot \frac{n}{2} \cdot \frac{n}{4} \cdot \frac{n}{8} \cdot \frac{n}{16} \cdot \dots \cdot \frac{n}{x} = 1$$

$$1 \cdot 1 \times 2 \cdot 1 \times 4 \cdot 1 \times 8 \cdot 1 \times 16 \cdot \dots \cdot 1 \times x = 1$$

$$\text{ans} = \log_2^n$$

\* 

```
for(int i=0; i<=n; i=i*3){  
    //statements  
}
```

$$\text{ans} = \text{infinity iterations}$$

\* 

```
for(int i=1; i<=n; i=i++){  
    if(i==n/2) break;  
}
```

$$\text{ans} = n/2 \text{ iteration}$$

\* 

```
while(n>0){  
    n/=3;  
}
```

$$\text{ans} = \log_3 n$$

## Finding No of iterations with Nested Loops

```
* for(int i=1; i<=10; i++){
    for(int j=1; j<=n; j++){
        //statements
    }
}
```

$$n + n + n + \dots + n$$

ans = 10n

| <u>i</u> | <u>j</u> | <u>total_iteration</u> |
|----------|----------|------------------------|
| 1        | [1, n]   | n iteration            |
| 2        | [1, n]   | n iteration            |
| 3        | [1, n]   | n iteration            |
| 4        | [1, n]   | n iteration            |
| ⋮        | ⋮        | ⋮                      |
| 10       | [1, n]   | n iteration            |

```
* for(int i=1; i<=n; i++){
    for(int j=1; j<=m; j++){
        //statements
    }
}
```

$$m + m + m + \dots + m$$

ans = n \* m

| <u>i</u> | <u>j</u> | <u>total_iteration</u> |
|----------|----------|------------------------|
| 1        | [1, m]   | m iteration            |
| 2        | [1, m]   | m iteration            |
| 3        | [1, m]   | m iteration            |
| ⋮        | ⋮        | ⋮                      |
| n        | [1, m]   | m iteration            |

```
* for(int i=1; i<=n; i++){
    for(int j=1; j<=n; j=j*2){
        //statements
    }
}
```

$$\log_2 n + \log_2 n + \dots + n \text{ times}$$

ans =  $n * \log_2 n$

| <u>i</u> | <u>j</u>   | <u>total_iteration</u> |
|----------|------------|------------------------|
| 1        | $\log_2 n$ | $\log_2 n$             |
| 2        | $\log_2 n$ | $\log_2 n$             |
| 3        | $\log_2 n$ | $\log_2 n$             |
| ⋮        | ⋮          | ⋮                      |
| n        | $\log_2 n$ | $\log_2 n$             |

```
* for(int i=1; i<=n; i++){
    for(int j=1; j<=i; j++){
        //statements
    }
}
```

$$\Rightarrow 1 + 2 + 3 + 4 + \dots + n$$

$$\Rightarrow \frac{n * (n+1)}{2} \Rightarrow \frac{n^2 + n}{2} \Rightarrow \frac{n^3}{2}$$

| <u>i</u> | <u>j</u> | <u>total_iteration</u> |
|----------|----------|------------------------|
| 1        | [1, 1]   | 1 iteration            |
| 2        | [1, 2]   | 2 iteration            |
| 3        | [1, 3]   | 3 iteration            |
| ⋮        | ⋮        | ⋮                      |
| n        | [1, n]   | n iteration            |

# Arithmetic Progression:

$$S = 4 \quad \xrightarrow{+3} \quad 7 \quad \xrightarrow{+3} \quad 10 \quad \xrightarrow{+3} \quad 13 \quad \xrightarrow{+3} \quad 16 \quad \xrightarrow{+3} \quad 19 \quad \xrightarrow{+3} \quad 22$$

\* difference between any two consecutive num is constant

$$S = 5 \quad 10 \quad 15 \quad 20 \quad 25 \quad 30$$

First 4 terms

$$1) 4 + 7 + 10 + 13 = 34$$

$$2) 5 + 10 + 15 + 20 = 50$$

First n terms

$$S = 4 + 7 + 10 + 13 + 16 + 19 + 22$$

$$\begin{aligned} a+d &+ a+nd-\frac{1}{2}d \\ 2a+nd-d \\ 2a+(n-1)d \end{aligned}$$

$$S = \overset{1}{a} + \overset{2}{a+d} + \overset{3}{a+2d} + \overset{4}{a+3d} + \dots + a+(n-2)d + \overset{n}{a+(n-1)d}$$

(+)

$$S = a+(n-1)d + a+(n-2)d + \dots + a+d + a$$

$$2S = 2a+(n-1)d + 2a+(n-1)d + 2a+(n-1)d \dots \dots 2a+(n-1)d$$

$$2S = n * 2a + (n-1)d$$

$$S = \frac{n}{2} * (2a + (n-1)d)$$

n - no of terms

a - First number

d - common difference.

$$\begin{aligned} n &= 4 & S &= \frac{4}{2} * (4 + 9) \\ a &= 4 \\ d &= 3 & S &= 2 * 17 = 34 \end{aligned}$$

$$\begin{aligned} n &= 4 & S &= \frac{4}{2} * (10 + 15) \\ a &= 5 \\ d &= 5 & S &= 2 * 25 = 50 \end{aligned}$$

$$\begin{aligned} n &= 7 & S &= \frac{7}{2} * (4 + 18) \\ a &= 2 \\ d &= 3 & S &= 3.5 * 22 = 77 \end{aligned}$$

$$2 + 5 + 8 + 11 + 14 + 17 + 20 = 77$$

# Geometric Progression

$$S = 5 \quad 10 \quad 20 \quad 40 \quad 80 \quad 160 \dots$$

$\times 2 \quad \times 2 \quad \times 2 \quad \times 2 \quad \times 2$

\* ratio between any two consecutive num is Constant

First n terms

$$\begin{array}{lcl}
 S = & 5 & 10 \quad 20 \quad 40 \dots \\
 \textcircled{1} \text{ --- } S = & a & ar \quad ar^2 \quad ar^3 \quad ar^4 \dots \\
 \textcircled{2} \text{ --- } S.r = & ar & ar^2 \quad ar^3 \quad ar^4 \dots ar^n
 \end{array}$$

$$\textcircled{2} - \textcircled{1} \quad Sr - S = ar^n - a$$

$$S(r-1) = a(r^n - 1)$$

$$S = a \times \frac{(r^n - 1)}{(r - 1)}$$

$a$  - First Number

$r$  - Common ratio

$n$  - no of terms

Ex1:  $5 + 10 + 20 + 40 = 75$

$a=5$

$r=2$   
 $n=4$   $S = \frac{5 \times (2^4 - 1)}{1} \Rightarrow \frac{5 \times (15)}{1} = 75$

why do we compare the algorithms in worst Case?

| $n$       | Algorithm-1<br>$100 \log_2 n$ | Algorithm-2<br>$n/10$ | Winner      |
|-----------|-------------------------------|-----------------------|-------------|
| $n=100$   | 600                           | 10                    | Algorithm 2 |
| $n=1000$  | 900                           | 100                   | Algorithm 2 |
| $n=13000$ | 1300                          | 1300                  | Draw        |
| $n=25000$ | 1400                          | 2500                  | Algorithm-1 |
| $n=10^9$  | 2900                          | $10^8$                | Algorithm-1 |

# Asymptotic Analysis of Algorithm (Big O Notations)

\* Analysing Performance of the algorithm over the large input

Comparison Order from lower to higher

$$\log(\log n) < \log n < \sqrt{\log n} < n < n \log n < n \sqrt{\log n} < n^2 < n^3 < 2^n < n! < n^n$$

$$\frac{\log_2(64)}{6}$$

$$\log_2 6$$

$$n=100 \\ \sqrt{n} = 10$$

Steps to Calculate Big O Notation

\* Calculate No of iterations as per your code

\* Remove lower order terms if it is in Addition or Subtraction.

\* Remove Constants

$$n^2 + n!$$

$$5, 10, 15$$

Examples:

$$* \cancel{4n} + n \log n + \cancel{5} \Rightarrow n + n \log n = O(n \log n)$$

$$* \cancel{4n \log n} + \cancel{3n \sqrt{\log n}} + \cancel{10^8} \Rightarrow n \log n + n \sqrt{\log n} = O(n \sqrt{\log n})$$

$$* \cancel{280} \Rightarrow O(1)$$

Why do we ignore lower order terms?

| input                                                    | Total Iteration<br>$n^2 + 10n$ | % of lower order Contribution                                      |
|----------------------------------------------------------|--------------------------------|--------------------------------------------------------------------|
| $n=10$                                                   | $100 + 100 = 200$              | $\frac{100}{200} \times 100 = 50\%$                                |
| $n=100$                                                  | $10^4 + 10^3$                  | $\frac{10^3}{10^4} \times 100 \approx 9\%$                         |
| $n=10^4$                                                 | $10^8 + 10^5$                  | $\frac{10^5}{10^8} \times 100 \approx \underline{\underline{0.1}}$ |
| over the large input lower order term is very very small |                                |                                                                    |

# how to find Space Complexity

```
* Func{
    int a; — 4 bytes
    int b; — 4 bytes
    int c; — 4 bytes
}
```

= 12 bytes

= S.C =  $O(1)$

```
* Func(int n){
    int Arr[10]; — 40 bytes X
    int a,b,c; — 12 bytes X
    long A[] = new long[n];
                                8*n
}
```

S.C =  $O(n)$

```
* Func(int n){
    int a=n; — 4 bytes
    int A[] = new int[n]; — 4*n
    int B[][] = new int[n][n]; — 4*n*n
}
```

= ~~4~~ + ~~4n~~ +  $4n^2$

S.C =  $O(n^2)$

```
* Func(int A[], int n){
    int ans=A[0]; — 4 byte
    for(int i=1; i<=n; i++){
        ans=Math.max(ans, A[i]);
    }
    return ans;
}
```

T.C =  $O(n)$

S.C =  $O(1)$

## Practice Questions:

```
* for(int i=1; i<=n; i+=2){
    print(i);
}
```

Time Complexity :  $O(n)$

Space Complexity :  $O(1)$

$n=20$   
i  
1 3 5 7 9 11 13 15 17 19  
no of iteration =  $\frac{n}{2}$

```
* for(int i=1; i<=100; i++){
    print(i);
}
```

Time Complexity :  $O(1)$

Space Complexity :  $O(1)$

$\begin{matrix} a & b \\ [1, 100] \end{matrix}$  =

$b-a+1$

$100-1+1 = 100$  iteration

```
* void solve(int n,int m){
    for(int i=1;i<=n;i++){
        print(i);
    }
    for(int j=1;j<=m;j++){
        print(j);
    }
}
```

Time Complexity :  $O(n+m)$

Space Complexity :  $O(1)$

$[1, n]$  -  $n$  iteration

$[1, m]$  -  $m$  iteration

$n+m$  iteration

$\Rightarrow O(\max(n, m))$

```
* for(int i=0;i<n;i++){
    for(int j=0;j<=i;j++){
        print(i+j);
    }
}
```

Time Complexity :  $O(n^2)$

Space Complexity :  $O(1)$

| <u>i</u> | <u>j</u> | <u>total</u> |
|----------|----------|--------------|
| 0        | [0,0]    | 1            |
| 1        | [0,1]    | 2            |
| 2        | [0,2]    | 3            |
| 3        | [0,3]    | 4            |
| ...      | ...      | ...          |
| n-1      | [0, n-1] | n-1          |
| n        | [0, n]   | n            |

$$S.n = n \times \frac{(n+1)}{2}$$

$$\Rightarrow \frac{n^2 + n}{2}$$

```
* for(int i=1;i<=n;i*=2){
    for(int j=1;j<=n;j++){
        //statements
    }
}
```

Time Complexity :  $O(\log_2 n \times n)$

Space Complexity :  $O(1)$

outer loop  $n=64$

1 2 4 8 16 32 64  
 $\log_2 n$

inner loop

1, 2, 3 ..... 64  $n$  iteration

$\log_2 n \times n$

```
* int a=0,i=n;
while(i>0){
    a+=i;
    i/=2;
}
```

Time Complexity :  $O(\log_2 n)$

Space Complexity :  $O(1)$

$n=64$   
 $i=64$

64 32 16 8 4 2 1 0

$\log_2 n$

```
* for(int i=1;i<=100;i=i*2){
    for(int j=1;j<=n;j++){
        //statements
    }
}
```

Time Complexity :  $O(1)$

Space Complexity :  $O(1)$

Outer Loop :  $\log_2 100 \Rightarrow 6$

inner Loop :  $n$

Overall :  $6 \times n$

```
* int solve(int n){
    int s=0;
    for(int i=1;i*i<=n){
        s=s+i;
    }
    return s;
}
```

Time Complexity :  $O(\sqrt{n})$

Space Complexity :  $O(1)$

$$i \leq n/i \Rightarrow i * i \leq n \Rightarrow i^2 \leq n$$

take sqrt both side

$$(i^2)^{1/2} \leq n^{1/2} \Rightarrow i \leq n^{1/2} \Rightarrow i \leq \sqrt{n}$$

$\sqrt{n}$

```
* int s=0;
for(int i=1;i*i*i<=n){
    s=s+i;
}
```

Time Complexity :  $O(\sqrt[3]{n})$

Space Complexity :  $O(1)$

$$i^3 \leq n$$

$$(i^3)^{1/3} \leq n^{1/3} \Rightarrow i \leq n^{1/3}$$

$$i \leq \sqrt[3]{n}$$

```
* while(n>0){
    n/=3;
}
```

Time Complexity :  $O(\log_3 n)$

Space Complexity :  $O(1)$

$$n \quad n/3 \quad n/3^2 \quad n/3^3 \quad \dots$$

$$\log_3 n$$

```
* for(int i=0;i<=n;i*=2){
    //statements;
}
```

Time Complexity :  $O(\infty)$

Space Complexity :  $O(1)$

$[0, n]$

o o o o o o .....

infinity iteration

```
* for(int i=1;i<=n;i++){
    for(int j=1;j<=3^i;j++){
        print(i+j);
    }
}
```

Time Complexity :  $O(3^n)$

Space Complexity :  $O(1)$

| <u>i</u> | <u>j</u>   | <u>total</u> |
|----------|------------|--------------|
| 1        | $[1, 3^1]$ | 3            |
| 2        | $[1, 3^2]$ | $3^2$        |
| 3        | $[1, 3^3]$ | $3^3$        |
| 4        | $[1, 3^4]$ | $3^4$        |
| $\vdots$ | $\vdots$   | $\vdots$     |
| n        | $[1, 3^n]$ | $3^n$        |

$$3 \quad 3^2 \quad 3^3 \quad 3^4 \dots 3^n$$

$\times 3 \quad \times 3 \quad \times 3$

$$r=3 \quad a+(r^n-1)$$

$$a=3 \quad (r-1)$$

$$\frac{3 \times 3^n - 1}{2}$$

```
* void solve(int n){
    while(i<n){
        int x=i;
        while(x>0){
            x--;
        }
        i++;
    }
}
```

Time Complexity :  $O(n^2)$

Space Complexity :  $O(1)$

outer

|          |          |          |
|----------|----------|----------|
| i=1      | X[1,1]   | 1        |
| i=2      | X[2,1]   | 2        |
| i=3      | X[3,1]   | 3        |
| $\vdots$ | $\vdots$ | $\vdots$ |
| i=n      | X[n,1]   | n        |

$$\Rightarrow n \times \frac{(n+1)}{2} \Rightarrow \frac{n^2 + n}{2}$$

```
* for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        break;
    }
}
```

Time Complexity :  $O(n)$

Space Complexity :  $O(1)$

\* inner loop not even run  
more than one time

n iteration

```
* for(int i=0;i<n;i++){
    for(int j=n;j>i;j--){
        print(i+j);
    }
}
```

Time Complexity :  $O(n^2)$

Space Complexity :  $O(1)$

| <u>i</u> | <u>j</u>  | <u>total</u> |
|----------|-----------|--------------|
| 0        | $[0, 1]$  | 10           |
| 1        | $[0, 2]$  | 9            |
| 2        | $[0, 3]$  | 8            |
| $\vdots$ | $\vdots$  | $\vdots$     |
| 9        | $[0, 10]$ | 1            |

n=10

$$10 + 9 + 8 + 7 \dots 1$$

$$n + n-1 + n-2 + \dots 1$$

$$n \times \frac{(n+1)}{2}$$

```
* while(n>0){
    n++;
    n-=2;
}
```

Time Complexity :  $O(n)$

Space Complexity :  $O(1)$

n=10      n=9      n=8  
n++ - 11      n=10  
n-=2 - 9      n=8

n iteration

```
* for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        print(i+j);
    }
    for(int k=0;k<n;k++){
        print(k);
    }
}
```

Time Complexity :  $O(n^2)$

Space Complexity :  $O(1)$

i loop = n iteration }  $n * n$   
j loop = n iteration

k loop = n iteration

$$n^2 + n$$

```
* for(int i=1;i<=2^n;i++){
    int j=i;
    while(j>=1){
        j--;
    }
}
```

Time Complexity :  $O(4^n)$

Space Complexity :  $O(1)$

| <u>i</u> | <u>j</u>   | <u>total</u> |
|----------|------------|--------------|
| 1        | [1, 1]     | 1            |
| 2        | [2, 1]     | 2            |
| 3        | [3, 1]     | 3            |
| $\vdots$ | $\vdots$   | $\vdots$     |
| $2^n$    | $[2^n, 1]$ | $2^n$        |

$= 4^n$

$1 + 2 + 3 + \dots + n$   
 $\frac{n * (n+1)}{2}$   
 $1 + 2 + 3 + \dots + 2^n$   
 $\frac{2^n * (2^n + 1)}{2}$

```
* for(int i=0;i<n;i++){
    for(int j=i-1;j>=0;j++){
        //statements
    }
}
```

Time Complexity :  $O(\infty)$

Space Complexity :  $O(1)$

outer loop = n iteration  $\times$

inner loop

i=1 j=0  $0 \geq 0$  infinity iteration  
 $1 \geq 0$   
 $2 \geq 0$   
 $3 \geq 0$

```
* for(int i=0;i<n;i++){
    for(int j=i;j<=n&& j>i;j++){
        //statements
    }
}
```

Time Complexity :  $O(n)$

Space Complexity :  $O(1)$

outer loop : n iteration

inner loop: always false  
 which means never execute

```
* while(n>0){
    n/=2;
    n*=2;
}
```

Time Complexity :  $O(\infty)$

Space Complexity :  $O(1)$

$$n = 64 > 0$$

$$32 \rightarrow 64$$

infinity iteration

\* Time Limit Exceeded

\* If an algorithm has a time complexity of  $\underline{O(1)}$  then complexity of it?

1. Constant
2. Polynomial
3. exponential
4. None of the above

\* If an algorithm has a time complexity of  $O(n)$  then complexity of it?

1. Constant
2. Polynomial
3. exponential
4. Linear

\* If for an algorithm time complexity as  $O(\log_2 n)$ , then time complexity will?

1. Constant
2. Exponential
3. Linear
4. None of the above

\* If for an algorithm time complexity as  $O((3/2)^n)$  then time complexity will?  $(1.5)^n$

1. Constant
2. Exponential
3. Linear
4. None of the above

---

\* Practice More Questions from Google/ChatGPT

\* It is very important to discuss Time & Space complexity if you are Preparing for Top Product based Companies.